Язык программирования C

Лекция 7: inline, variadic functions, статические и динамические библиотеки

Лисица Никита

2026

Инлайнинг

- В коде часто встречаются маленькие вспомогательные функции:

```
int max(int x, int y) {
  return (x < y) ? y : x;
}
```

```
float sqr(float x) {
  return x * x;
}
```

Инлайнинг

- В коде часто встречаются маленькие вспомогательные функции:

```
int max(int x, int y) {
  return (x < y) ? y : x;
}
```

```
float sqr(float x) {
  return x * x;
}
```

- При вызове такой функции больше времени тратится на сам вызов, чем на исполнение кода функции

Инлайнинг

- *Инлайнинг (inlining)* – оптимизация компилятора, при которой код функции подставляется целиком в место её вызова (in-line)

Инлайнинг

- *Инлайнинг (inlining)* – оптимизация компилятора, при которой код функции подставляется целиком в место её вызова (in-line)

```
// Было
int a = 10, b = 20;
int x = max(a, b);
```

```
// Стало
int a = 10, b = 20;
int x = (a < b) ? b : a;
```

Инлайнинг

- *Инлайнинг (inlining)* – оптимизация компилятора, при которой код функции подставляется целиком в место её вызова (in-line)

```
// Было
int a = 10, b = 20;
int x = max(a, b);
```

```
// Стало
int a = 10, b = 20;
int x = (a < b) ? b : a;
```

- Компилятор **сам решает**, когда делать эту оптимизацию (зависит от включенного уровня оптимизации)

Инлайнинг

- *Инлайнинг (inlining)* – оптимизация компилятора, при которой код функции подставляется целиком в место её вызова (in-line)

```
// Было
int a = 10, b = 20;
int x = max(a, b);
```

```
// Стало
int a = 10, b = 20;
int x = (a < b) ? b : a;
```

- Компилятор **сам решает**, когда делать эту оптимизацию (зависит от включенного уровня оптимизации)
- Можно форсировать специфичными для компилятора механизмами: `__attribute__((always_inline))` в GCC, `__forceinline` в MSVC

Инлайнинг

- Мы привыкли объявлять функции в заголовочном файле и описывать их код в `.c`-файле

```c
// max.h
int max(int a, int b);
```

```c
// max.c
int max(int x, int y) {
  return (x < y) ? y : x;
}
```

Инлайнинг

- Мы привыкли объявлять функции в заголовочном файле и описывать их код в `.c`-файле

```c
// max.h
int max(int a, int b);
```

```c
// max.c
int max(int x, int y) {
  return (x < y) ? y : x;
}
```

- Сможет ли компилятор в этом случае заинлайнить вызов функции вне файла `max.c`?

Инлайнинг

- Мы привыкли объявлять функции в заголовочном файле и описывать их код в `.c`-файле

```c
// max.h
int max(int a, int b);
```

```c
// max.c
int max(int x, int y) {
  return (x < y) ? y : x;
}
```

- Сможет ли компилятор в этом случае заинлайнить вызов функции вне файла `max.c`? Нет – для инлайнинга нужен доступ к коду, а не только к объявлению

Инлайнинг

- Решение: поместим код функции прямо в заголовочный файл:

```
// max.h
int max(int a, int b) {
  return (x < y) ? y : x;
}
```

Инлайнинг

- Решение: поместим код функции прямо в заголовочный файл:

```
// max.h
int max(int a, int b) {
  return (x < y) ? y : x;
}
```

- Есть ли с этим проблемы?

Инлайнинг

- Решение: поместим код функции прямо в заголовочный файл:

```
// max.h
int max(int a, int b) {
  return (x < y) ? y : x;
}
```

- Есть ли с этим проблемы? Да: multiple definition, если `max.h` заинклюдят хотя бы два других файла

```
// a.c
#include <max.h>

...
```

```
// b.c
#include <max.h>

...
```

Атрибут inline

- Чтобы обойти эту проблему, у функций есть особый атрибут **`inline`**:

```
// max.h
inline int max(int a, int b) {
  return (x < y) ? y : x;
}
```

Атрибут inline

- Чтобы обойти эту проблему, у функций есть особый атрибут `inline`:

```
// max.h
inline int max(int a, int b) {
  return (x < y) ? y : x;
}
```

- `inline` – указание **линковщику**, что multiple definition этой конкретной фукнции – **не ошибка**, и он может взять любую

Атрибут inline

- Чтобы обойти эту проблему, у функций есть особый атрибут `inline`:

```
// max.h
inline int max(int a, int b) {
  return (x < y) ? y : x;
}
```

- `inline` – указание **линковщику**, что multiple definition этой конкретной фукнции – **не ошибка**, и он может взять любую
- Если у вас есть несколько *разных* `inline`-функций с одинаковым названием, это undefined behavior

Атрибут inline

- $\Longrightarrow$ Если вы хотите, чтобы функция инлайнилась, нужно
  1. Положить её реализацию в заголовочный файл, и
  2. Пометить её атрибутом **`inline`**, чтобы не ругался линковщик

Атрибут inline

- $\Longrightarrow$ Если вы хотите, чтобы функция инлайнилась, нужно
  1. Положить её реализацию в заголовочный файл, и
  2. Пометить её атрибутом **`inline`**, чтобы не ругался линковщик
- **N.B.:** На решение компилятора делать инлайнинг атрибут **`inline`** **не влияет**, он сам разберётся :)

Атрибут inline

- $\Longrightarrow$ Если вы хотите, чтобы функция инлайнилась, нужно
  1. Положить её реализацию в заголовочный файл, и
  2. Пометить её атрибутом **`inline`**, чтобы не ругался линковщик
- **N.B.:** На решение компилятора делать инлайнинг атрибут **`inline`** **не влияет**, он сам разберётся :)
- **`inline`** – флаг именно для линковщика

Variadic functions

- Мы видели функции, принимающие произвольное количество аргументов: `printf`, `scanf`, и т.п. Как это работает?

Variadic functions

- Мы видели функции, принимающие произвольное количество аргументов: `printf`, `scanf`, и т.п. Как это работает?
- В объявлении таких функций после списка аргументов стоит *многочие*, соответствующее *variadic* аргументам:

```
int printf(const char *format, ...);
```

Variadic functions

- Мы видели функции, принимающие произвольное количество аргументов: `printf`, `scanf`, и т.п. Как это работает?
- В объявлении таких функций после списка аргументов стоит *многочие*, соответствующее *variadic* аргументам:

```
int printf(const char *format, ...);
```

- Концептуально: variadic аргументы функции кладутся на стек, и ожидается, что функция сама знает их количество и типы (например, `printf` знает это из строки формата)

Variadic functions

- Мы видели функции, принимающие произвольное количество аргументов: `printf`, `scanf`, и т.п. Как это работает?
- В объявлении таких функций после списка аргументов стоит *многочие*, соответствующее *variadic* аргументам:

```
int printf(const char *format, ...);
```

- Концептуально: variadic аргументы функции кладутся на стек, и ожидается, что функция сама знает их количество и типы (например, `printf` знает это из строки формата)
- На практике аргументы передаются частично через регистры и частично через стек (см. ABI и calling conventions конкретной платформы)

Variadic functions

- Мы видели функции, принимающие произвольное количество аргументов: `printf`, `scanf`, и т.п. Как это работает?
- В объявлении таких функций после списка аргументов стоит *многочие*, соответствующее *variadic* аргументам:

```
int printf(const char *format, ...);
```

- Концептуально: variadic аргументы функции кладутся на стек, и ожидается, что функция сама знает их количество и типы (например, `printf` знает это из строки формата)
- На практике аргументы передаются частично через регистры и частично через стек (см. ABI и calling conventions конкретной платформы)
- **N.B.:** В C++ мы встретим *variadic templates* – это похожий по общее идее, но совершенно другой по реализации механизм :)

Variadic functions

- Для реализации variadic функций есть специальные типы и функции (на самом деле – макросы):

Variadic functions

- Для реализации variadic функций есть специальные типы и функции (на самом деле – макросы):
  - `va_start()`, `va_end()` – начать и закончить обработку аргументов

Variadic functions

- Для реализации variadic функций есть специальные типы и функции (на самом деле – макросы):
  - `va_start()`, `va_end()` – начать и закончить обработку аргументов
  - `va_arg()` – получить текущий аргумент и перейти к следующему

Variadic functions

- Для реализации variadic функций есть специальные типы и функции (на самом деле – макросы):
  - `va_start()`, `va_end()` – начать и закончить обработку аргументов
  - `va_arg()` – получить текущий аргумент и перейти к следующему
  - `va_list` – объект, хранящий состояние прохода по аргументам

Variadic functions

- Пример variadic функции, реализующей сумму произвольного набора чисел (первый аргумент – количество):

```
int sum(int count, ...) {
  int result = 0;
  va_list args;
  va_start(args, count);
  while (count > 0) {
    result += va_arg(args, int);
  }
  va_end(args);
  return result;
}
```

Variadic functions

- Пример variadic функции, реализующей примитивный printf:

```
void printf(const char *format, ...) {
  va_list args;
  va_start(args, format);
  while (*format != '\0') {
    if (*format == '%') {
      ++format;
      if (*format == 'd') {
        int value = va_arg(args, int);
        // Печатаем int
      } else if (*format == 's') {
        const char * value = va_arg(args, const char *);
        // Печатаем строку
      }
    } else {
      char c = *format;
      // Печатаем char
    }
    ++format;
  }
  va_end(args);
}
```

Variadic functions: проблемы

- Трудно обнаружить ошибки несоответствия типов/количества аргументов:

```
// Все эти вызовы - undefined behavior:
printf("%d %d", 10);
printf("%d", 11, 12);
printf("%s", 3.1415f);
```

Variadic functions: проблемы

- Трудно обнаружить ошибки несоответствия типов/количества аргументов:

```
// Все эти вызовы - undefined behavior:
printf("%d %d", 10);
printf("%d", 11, 12);
printf("%s", 3.1415f);
```

- Современные компиляторы и IDE знают про библиотечные variadic функции (`printf` и т.п.) и выдают предупреждения при несоответствии типов/количества аргументов

Variadic functions: проблемы

- Трудно обнаружить ошибки несоответствия типов/количества аргументов:

```
// Все эти вызовы - undefined behavior:
printf("%d %d", 10);
printf("%d", 11, 12);
printf("%s", 3.1415f);
```

- Современные компиляторы и IDE знают про библиотечные variadic функции (`printf` и т.п.) и выдают предупреждения при несоответствии типов/количества аргументов
- Механизм *variadic templates* в C++ (про который мы узнаем позже) лишён этого недостатка

Виды библиотек

- *Библиотека (library)* – набор кода, предназначенного для переиспользования в разных программах/проектах

Виды библиотек

- *Библиотека (library)* – набор кода, предназначенного для переиспользования в разных программах/проектах
- Они различаются по тому, как мы их подключаем к нашему коду:

Виды библиотек

- *Библиотека (library)* – набор кода, предназначенного для переиспользования в разных программах/проектах
- Они различаются по тому, как мы их подключаем к нашему коду:
  - *Header-only* библиотеки

Виды библиотек

- *Библиотека (library)* – набор кода, предназначенного для переиспользования в разных программах/проектах
- Они различаются по тому, как мы их подключаем к нашему коду:
  - *Header-only* библиотеки
  - *Статический (static)* библиотеки

Виды библиотек

- *Библиотека (library)* – набор кода, предназначенного для переиспользования в разных программах/проектах
- Они различаются по тому, как мы их подключаем к нашему коду:
  - *Header-only* библиотеки
  - *Статический (static)* библиотеки
  - *Динамические (dynamic)* библиотеки

Header-only библиотеки

- *Header-only* библиотеки состоят только из заголовочных файлов

Header-only библиотеки

- *Header-only* библиотеки состоят только из заголовочных файлов
- Для их подключения достаточно сказать компилятору правильный путь до заголовочных файлов (`-I` в GCC, и т.п.)

Header-only библиотеки

- *Header-only* библиотеки состоят только из заголовочных файлов
- Для их подключения достаточно сказать компилятору правильный путь до заголовочных файлов (`-I` в GCC, и т.п.)
- Проще всего для добавления в существующую систему сборки

Header-only библиотеки

- *Header-only* библиотеки состоят только из заголовочных файлов
- Для их подключения достаточно сказать компилятору правильный путь до заголовочных файлов (`-I` в GCC, и т.п.)
- Проще всего для добавления в существующую систему сборки
- Если библиотека используется несколькими программами, её приходится компилировать каждый раз заново

Header-only библиотеки: пример

```c
// vector.h

typedef struct {
  float x, y, z;
} vec3;

vec3 add(vec3 a, vec3 b) {
  return (vec3){a.x + b.x, a.y + b.y, a.z + b.z};
}

vec3 muls(vec3 a, float b) {
  return (vec3){a.x * b, a.y * b, a.z * b};
}
```

Header-only библиотеки: пример

```c
// main.c
#include <vector.h>

int main() {
  vec3 v = {1.f, 2.f, 3.f};
  vec3 u = add(v, muls(v, 0.5f));
  return 0;
}
```

```
gcc main.c -o main -I path/to/vector/includes
```

Статические библиотеки

- *Статические (static)* библиотеки состоят из заголовочных и `.c` файлов

Статические библиотеки

- *Статические (static)* библиотеки состоят из заголовочных и `.c` файлов
- Заголовочные файлы описывают *публичный интерфейс* библиотеки (функции, через которые мы пользуемся этой библиотекой)

Статические библиотеки

- *Статические (static)* библиотеки состоят из заголовочных и `.c` файлов
- Заголовочные файлы описывают *публичный интерфейс* библиотеки (функции, через которые мы пользуемся этой библиотекой)
- `.c` файлы компилируются в бинарный файл библиотеки (`.a` в Linux, `.lib` в Windows)

Статические библиотеки

- *Статические (static)* библиотеки состоят из заголовочных и `.c` файлов
- Заголовочные файлы описывают *публичный интерфейс* библиотеки (функции, через которые мы пользуемся этой библиотекой)
- `.c` файлы компилируются в бинарный файл библиотеки (`.a` в Linux, `.lib` в Windows)
- При **линковке нашей программы** статическая библиотека *склеивается* с нашей программой

Статические библиотеки

- *Статические (static)* библиотеки состоят из заголовочных и `.c` файлов
- Заголовочные файлы описывают *публичный интерфейс* библиотеки (функции, через которые мы пользуемся этой библиотекой)
- `.c` файлы компилируются в бинарный файл библиотеки (`.a` в Linux, `.lib` в Windows)
- При **линковке нашей программы** статическая библиотека *склеивается* с нашей программой
- При обновлении библиотеки придётся перекомпилировать программу

Статические библиотеки: пример

```c
// vector.h

typedef struct {
  float x, y, z;
} vec3;

vec3 add(vec3 a, vec3 b);
vec3 muls(vec3 a, float b);
```

```c
// vector.c
#include <vector.h>

vec3 add(vec3 a, vec3 b) {
  return (vec3){a.x + b.x, a.y + b.y, a.z + b.z};
}

vec3 muls(vec3 a, float b) {
  return (vec3){a.x * b, a.y * b, a.z * b};
}
```

Статические библиотеки: пример

```
# Компиляция библиотеки
gcc -c vector.c -o vector.o \
  -I path/to/vector/includes

# "Архивация библиотеки"
ar rcs libvector.a vector.o

# Компиляция программы
gcc main.c -o main -lvector \
  -I path/to/vector/includes \
  -L path/to/vector/lib
```

Динамические библиотеки

- *Динамические (static)* библиотеки компилируются независимо от нашей программы и лежат в отдельных файлах (`.so` в Linux, `.dll` в Windows, `.dylib` в macOS)

Динамические библиотеки

- *Динамические (static)* библиотеки компилируются независимо от нашей программы и лежат в отдельных файлах (`.so` в Linux, `.dll` в Windows, `.dylib` в macOS)
- Могут устанавливаться вместе с программой, а могут поставляться системой (и устанавливаться напр. через менеджеры пакетов)

Динамические библиотеки

- *Динамические (static)* библиотеки компилируются независимо от нашей программы и лежат в отдельных файлах (`.so` в Linux, `.dll` в Windows, `.dylib` в macOS)
- Могут устанавливаться вместе с программой, а могут поставляться системой (и устанавливаться напр. через менеджеры пакетов)
- Наша программа **при запуске** загружает нужные динамические библиотеки в своё адресное пространство
  $\Longrightarrow$ программе нужно как-то узнать, где они лежат (в Linux за это отвечает механизм RPATH)

Динамические библиотеки

- Вызов функции из динамической библиотеки чуть дороже, чем вызов обычной функции (см. Procedure Linkage Table и Global Offset Table в ELF)

Динамические библиотеки

- Вызов функции из динамической библиотеки чуть дороже, чем вызов обычной функции (см. Procedure Linkage Table и Global Offset Table в ELF)
- При изменении кода (но не публичного интерфейса!) динамической библиотеки достаточно просто перекомпилировать саму библиотеку – использующие её программы автоматически загрузят новую версию при запуске

Динамические библиотеки

- Вызов функции из динамической библиотеки чуть дороже, чем вызов обычной функции (см. Procedure Linkage Table и Global Offset Table в ELF)
- При изменении кода (но не публичного интерфейса!) динамической библиотеки достаточно просто перекомпилировать саму библиотеку – использующие её программы автоматически загрузят новую версию при запуске
- Почти всегда системные библиотеки, стандартная библиотека C, и т.п. – динамические

Динамические библиотеки: пример

```
# Компиляция библиотеки
gcc -fPIC -c vector.c -o vector.o \
  -I path/to/vector/includes

# Линковка библиотеки
gcc -shared vector.o -o libvector.so

# Компиляция программы
gcc main.c -o main -lvector \
  -I path/to/vector/includes \
  -L path/to/vector/lib \
  -Wl,-rpath,path/to/vector/lib
```